



Curso: Programação Full Stack

Processo de Desenvolvimento de Software

Prof. Francisco Davi



Uma empresa contratou vocês para desenvolver o sistema web de uma barbearia. São quatro desenvolvedores. Todos começam a programar imediatamente.

➡ **Não existem tarefas definidas, prioridades, responsável por cada atividade nem prazo para as funcionalidades.**

“O que provavelmente vai dar errado?”

?

?

?

?

?

?

?

?

?

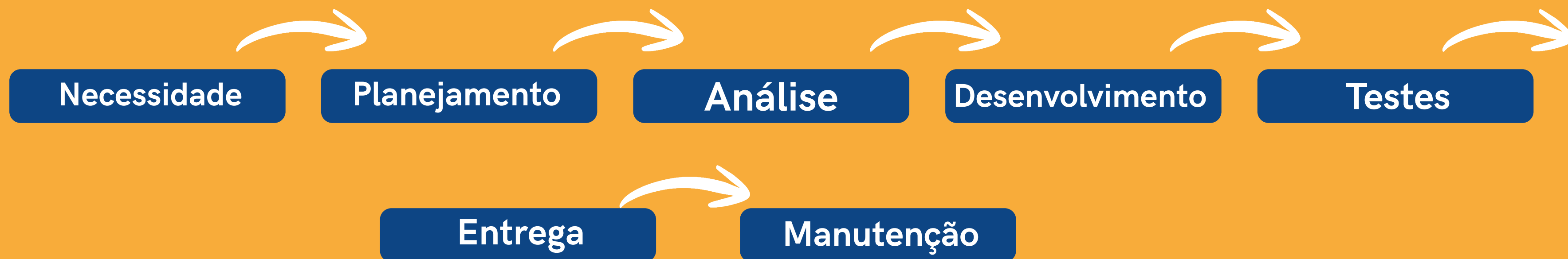
PROCESSO → organização de como o software será desenvolvido.





Processo de desenvolvimento de software

Não se começa simplesmente a programar



Isso já está acontecendo na UC

ICE BOX

In progress

Emergency

Testing

Complete

Nas aulas anteriores fizemos briefing, público-alvo, análise de requisitos etc. Percebam que ainda nem começamos a programar. Tudo isso faz parte do processo de construir software."

Computer System Administrator

Quality Assurance Person

ISO 9001 blank DVD

Software Tester

Technical Writer

Analysis

Computer Support Specialist

Process Improvement

Information Resource Specialist

Optimization

Information Resource Specialist

Software Tester

New football !!

Network Architecture

C++ Java installation

Database Administrator finding

add status of tasks in web interface

Processos ágeis

O problema que o ágil enfrenta

O problema que o ágil enfrenta

Imagine um projeto planejado para durar 8 meses. A equipe passa meses desenvolvendo e só no final apresenta tudo ao cliente.

"Não era bem isso que eu queria."

E se os ciclos forem menores: planejar, desenvolver, entregar, receber feedback, melhorar — e repetir.

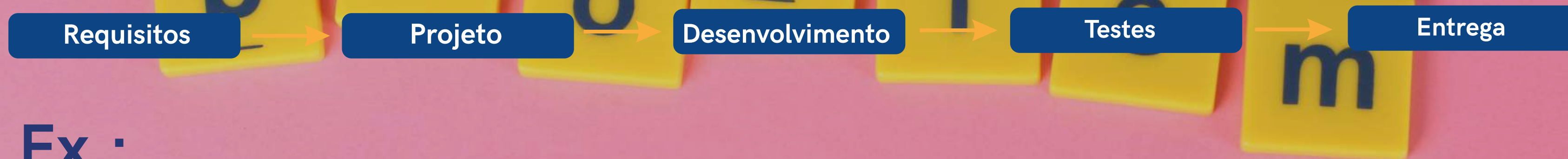


Como surgiu?

➡ Como uma resposta aos problemas que equipes enfrentavam com processos muito rígidos e pesados, especialmente quando os requisitos mudavam durante o projeto.

Antes do Agil

➡ Durante décadas, era comum organizar projetos de software de maneira bastante sequencial:



Ex.:

➡ Um exemplo clássico é o modelo em **Cascata (Waterfall)**.

E aí começaram outras abordagens

Durante as décadas de 1980 e 1990, várias ideias começaram a ganhar força buscando desenvolvimento mais iterativo, incremental e adaptável.

Entre elas surgiram ou se consolidaram abordagens como:

➔ Scrum, Extreme Programming (XP), DSDM, Crystal

O Ágil não nasceu primeiro como uma metodologia única.



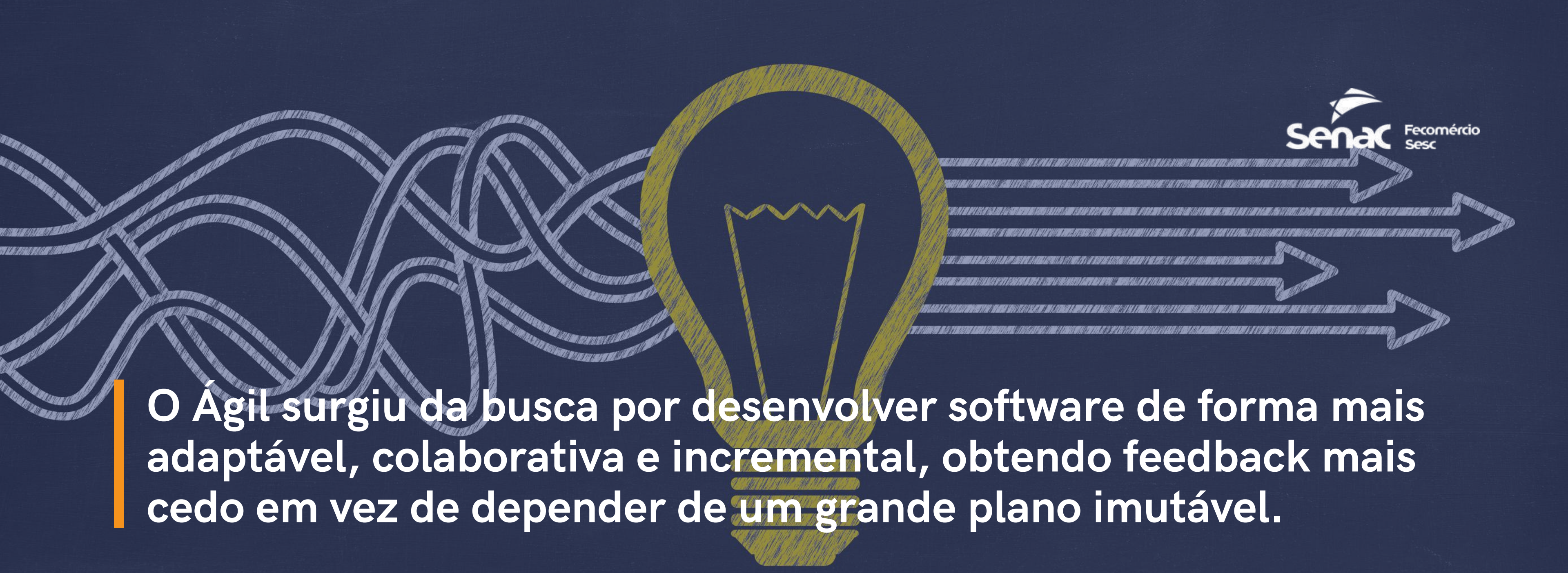
Manifesto para Desenvolvimento Ágil de Software

➔ Manifesto Ágil.

Indivíduos e interações mais que processos e ferramentas
Software em funcionamento mais que documentação abrangente
Colaboração com o cliente mais que negociação de contratos
Responder a mudanças mais que seguir um plano

O manifesto não diz:

- ✗ Não faça documentação.
- ✗ Não faça planejamento.
- ✗ Não use processos.



O Ágil surgiu da busca por desenvolver software de forma mais adaptável, colaborativa e incremental, obtendo feedback mais cedo em vez de depender de um grande plano imutável.



Processos ágeis

O problema não era simplesmente o Cascata ser “ruim”. Ele pode funcionar bem em contextos mais previsíveis. A dificuldade aparece quando o projeto tem muita incerteza e mudanças frequentes.

Imagine:

Uma empresa pede um sistema.
A equipe levanta os requisitos e planeja tudo.
Passa meses desenvolvendo.
Quando o cliente finalmente vê o produto funcionando, diz:
“Minhas necessidades mudaram.”

➡ Quanto mais tarde essa descoberta acontece, mais retrabalho pode existir.

Preditivo × Ágil

Abordagem preditiva

- ➔ Planejamento mais antecipado
- ➔ Mudanças tendem a ser mais custosas
- ➔ Entregas podem ser maiores
- ➔ Feedback pode demorar mais

vs

Abordagem ágil

- ➔ Planejamento adaptativo
- ➔ Mudanças são esperadas
- ➔ Entregas frequentes e incrementais
- ➔ Feedback frequente

Ágil não significa "fazer rápido" nem "fazer sem documentação". Ágil = entregar valor em pequenos ciclos, aprender com o feedback e adaptar o trabalho.

Extreme Programming

XP: uma abordagem ágil orientada à prática

➡ Pair Programming

Dois desenvolvedores trabalhando juntos: um programa enquanto o outro acompanha, questiona e revisa.

➡ TDD

Teste → código → refatoração.
Primeiro define-se o comportamento esperado, depois escreve-se o código necessário.

➡ Refatoração

Melhorar internamente o código sem alterar seu comportamento externo.

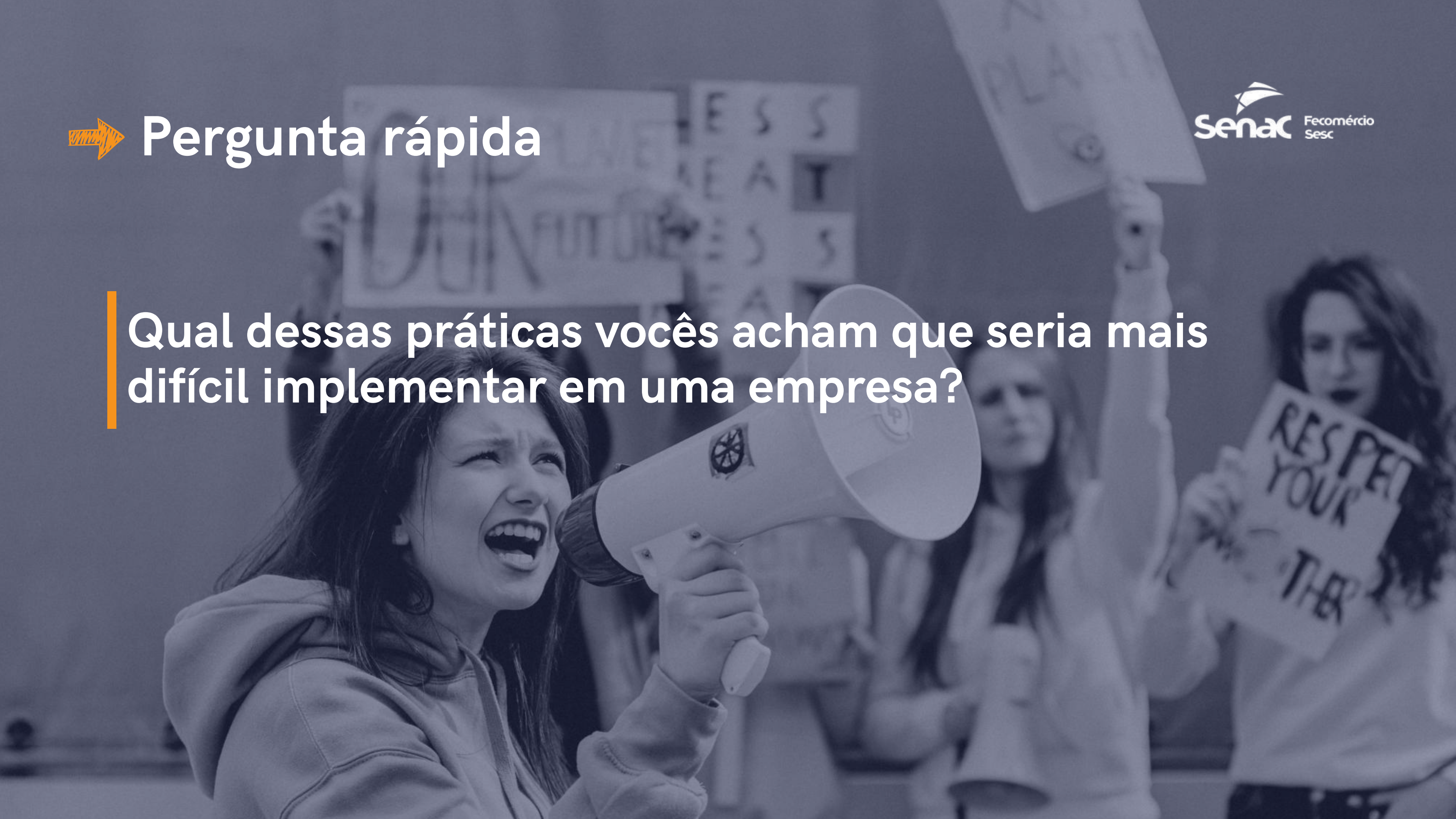
➡ Integração contínua

Integrar alterações frequentemente para identificar problemas rapidamente.

➡ Pequenas entregas

Evitar esperar meses para apresentar algo funcionando.

BEST
PRACTICE

A grayscale photograph of a woman in the foreground, shouting into a megaphone. In the background, other people are visible, some holding signs. One sign clearly says "RESPECT YOUR TEAM".

➡ Pergunta rápida

Qual dessas práticas vocês acham que seria mais difícil implementar em uma empresa?

O Scrum

➡ O ciclo do Scrum

Product Backlog

Sprint

Trabalho

Incremento

Feedback

Próxima Sprint

Sprint — período de duração fixa em que a equipe trabalha para atingir um objetivo e produzir um incremento útil do produto.

O Scrum

➡ Responsabilidades no Scrum

➡ Product Owner

Maximiza o valor do produto e gerencia efetivamente o Product Backlog.

➡ Scrum Master

Ajuda a equipe e a organização a compreender e aplicar Scrum, aumentando a efetividade do time.

➡ Developers

Profissionais responsáveis por criar um incremento utilizável a cada Sprint.

O Scrum

➡ Os quatro eventos

➡ Sprint Planning

"O que vamos buscar nesta Sprint e como faremos?"

➡ Daily Scrum

"Como estamos avançando em direção ao objetivo?"

➡ Sprint Review

"O que construímos e que feedback recebemos?"

➡ Retrospective

"Como podemos trabalhar melhor na próxima Sprint?"

➡ **Review ≠ Retrospective.** A Review olha para o produto e para o Product Backlog. A Retrospective olha para como a equipe trabalhou e como melhorar sua efetividade.

Cliente, Usuário e Público-Alvo



Cliente

Quem paga pelo desenvolvimento do site e toma as decisões de negócio (ex: O dono da barbearia).



Público-Alvo

O perfil demográfico e comportamental que o negócio deseja atrair (ex: Homens entre 20 e 45 anos, residentes na região central, interessados em cuidados pessoais)



Usuário

Quem efetivamente interage com a interface do sistema (ex: O cliente da barbearia agendando um corte, ou o recepcionista gerenciando a agenda no painel).

O Kanban

➡ Visualizar o fluxo de trabalho

● A fazer

Agendamento

Serviços

● Em andamento

Página inicial

● Teste

Cadastro

● Concluído

Header

"Só de olhar esse quadro, o que vocês conseguem descobrir sobre o projeto?"

O Kanban

➡ Limite de WIP e gargalos

Work in Progress (WIP) é o trabalho em andamento. Limitar o WIP ajuda a acompanhar o fluxo e identificar gargalos.

15

A fazer

8

Em desenvolvimento

12

Teste

2

Concluído

Onde parece estar o gargalo?

Organizando o Sistema da Barbearia

- ➔ Transformar as funcionalidades em um pequeno Product Backlog
- ➔ Escolher algumas para uma Sprint e montar um quadro: Backlog → A fazer → Em andamento → Teste → Concluído
- ➔ Definir quem assume Product Owner, Scrum Master e Developers na simulação



Um imprevisto do cliente

 O cliente entrou em contato e informou que o agendamento pelo celular passou a ser prioridade máxima.

Como uma equipe ágil lidaria com essa mudança?



Referências

- AGNER, L. Ergodesign e Arquitetura de Informação: Trabalhando com o Usuário. São Paulo: Senac, 2018.
- GEORGE, J.; BEAIRD, J. Princípios do Web Design Maravilhoso. Rio de Janeiro: Alta Books, 2016.
- DIAS, P. R. P. Microsoft Project 2016. São Paulo: Senac, 2016.
- JOYCE, P.; SILLS, C. Técnicas em Gestalt: aconselhamento e psicoterapia. São Paulo: Vozes, 2016

